

User Manual

for S32 PLATFORM Driver

Document Number: UM11PLATFORMASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	7
3.1 Requirements	7
3.2 Driver Design Summary	7
3.3 Hardware Resources	8
3.4 Deviations from Requirements	9
3.5 Driver Limitations	9
3.6 Driver usage and configuration tips	9
3.6.1 Initialization	9
3.6.2 Handling the interrupts	10
3.7 Runtime errors	10
3.8 Symbolic Names Disclaimer	10
4 Tresos Configuration Plug-in	11
4.1 Module Platform	12
4.2 Container GeneralConfiguration	12
4.3 Parameter PlatformDevErrorDetect	13
4.4 Parameter PlatformMcmConfigurable	13
4.5 Parameter PlatformIpAPIsAvailable	14
4.6 Parameter PlatformEnableUserModeSupport	14
4.7 Parameter PlatformMulticoreSupport	15
4.8 Parameter PlatformEnableVtorConfiguration	15
4.9 Reference PlatformEcucPartitionRef	16
4.10 Container McmConfig	16
4.11 Parameter SystemAhbSlavePrio	16
4.12 Reference PlatformMcmEcucPartitionRef	17
4.13 Container SystemIsrConfig	17
4.14 Parameter SystemIsrName	18
4.15 Parameter SystemIsrEnabled	18
4.16 Container IntCtrlConfig	19
4.17 Parameter PlatformVtorAddressConfig	19
4.18 Reference PlatformNvicEcucPartitionRef	20
4.19 Container PlatformIsrConfig	20

4.20 Parameter IsrName	21
4.21 Parameter IsrEnabled	23
4.22 Parameter IsrPriority	23
4.23 Parameter IsrHandler	24
4.24 Container CommonPublishedInformation	24
4.25 Parameter ArReleaseMajorVersion	24
4.26 Parameter ArReleaseMinorVersion	25
4.27 Parameter ArReleaseRevisionVersion	25
4.28 Parameter ModuleId	26
4.29 Parameter SwMajorVersion	26
4.30 Parameter SwMinorVersion	27
4.31 Parameter SwPatchVersion	27
4.32 Parameter VendorApiInfix	28
4.33 Parameter VendorId	28
5 Module Index	30
5.1 Software Specification	30
6 Module Documentation	31
6.1 Interrupt Controller IP	31
6.1.1 Detailed Description	31
6.1.2 Data Structure Documentation	32
6.1.3 Types Reference	34
6.1.4 Enum Reference	34
6.1.5 Function Reference	36
6.2 Platform	41
6.2.1 Detailed Description	41
6.2.2 Data Structure Documentation	42
6.2.3 Macro Definition Documentation	42
6.2.4 Types Reference	46
6.2.5 Function Reference	46
6.3 System IP	50
6.3.1 Detailed Description	50
6.3.2 Function Reference	50



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes the NXP Semiconductor PLATFORM driver for S32. The PLATFORM driver configuration parameters and deviations from the specification are described in PLATFORM Driver chapter of this document. PLATFORM driver requirements and APIs are vendor-specific.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
IRQ	Interrupt request
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	General Specification of Basic Software Modules	AUTOSAR Release 4.4.0
2	Specification of Communication Stack Types	AUTOSAR Release 4.4.0
3	Specification of Compiler Abstraction	AUTOSAR Release 4.4.0
4	Specification of Platform Types	AUTOSAR Release 4.4.0
5	Specification of Standard Types	AUTOSAR Release 4.4.0
6	S32G2 Reference Manual	Rev. 7, February 2023
7	S32G3 Reference Manual	Rev. 3, Feb 2023
8	S32R45 Reference Manual	Rev. 3, 12/2021
9	S32G2 Data Sheet	Rev. 6, Dec 2022
10	S32G3 Data Sheet	Rev. 2 RC, Feb 2023
11	S32R45 Data Sheet	Rev. 3 RC, 11/2022
12	VR5510 Data Sheet	Rev. 5 RC, April 2022
13	S32G2 Errata Document	Mask Set Errata for Mask 0P77B, v3.0

Introduction

#	Title	Version
14	S32G2 Errata Document	Mask Set Errata for Mask 1P77B, v1.0
15	S32G3 Errata Document	Mask Set Errata for Mask 1P72B, v2.1
16	S32R45 Errata Document	Mask Set Errata for Mask P57D, Rev. 2.0

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

PLATFORM is a complex driver, so there are no AUTOSAR requirements specific to this module. For the S32 platform, the PLATFORM module configures the interrupt controller and MCM functionalities. It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The PLATFORM driver configures platform specific settings, managing the interrupt requests and other system wide settings as defined in each hardware implementation.

Interrupt Controller

The configuration contains the list of interrupt requests, as defined per each platform; the application can enable the interrupts and set priorities. There is one Interrupt Controller belonging to each M7 core.

System Settings

On S32, PLATFORM driver allows the configuration of core-related interrupt requests, as defined in the implementation of MCM module, as well as other system specific settings (e.g. AHB slave access priority)

Linker files On S32, Internal RAM is different between these derivatives, linker file added more details information about memory size for each derivative. As currently, the linker has allocated memory for `int_dtcn`, `int_dtcn_stack`, `int_sram_shareable`, `int_sram`, `int_sram_no_cacheable`, it is used for the common S32XX so that user can refer to allowcate for all derivatives.

Below you can find the descriptions for each file present in the Platform module:

File Name	File Type	Description
nvic.h	Stub file. Must be replaced by all integrators.	This file is a stub. This file include set priority grouping, enable , disable, set priority interrupt.
sys_init.h	Stub file. Must be replaced by all integrators.	this file is a stub. This file include some functions Function used to disable the interrupt number id. Function used to enable the interrupt number id and set up the priority. Function used to register the interrupt handler in the interrupt vectors. Function used to enable all interrupts. Function used to disable all interrupts. Function used to initiaialize clocks, system clock is system Pll 120 MHz. Function used to enter halt mode. Function used to enter stop mode. Function used to provide the CoreID to EUnit.
system.h	Stub file. Must be replaced by all integrators.	This file is a stub. This file include define some macros SCB Interrupt Control State Register Definitions.
core_↔ specific.h	Stub file. Must be replaced by all integrators.	This file is a stub. This file necessary for mpu memory region configuration.
nvic.c	Stub file. Must be replaced by all integrators.	This file is a stub. Set Priority Grouping. The function sets the priority grouping field using the required unlock sequence. The parameter PriorityGroup is assigned to the field SCB->AICR [10:8] PRIGROUP field.
sys_init.c	Stub file. Must be replaced by all integrators.	This file is a stub. This file include some functions need to initialize the clock.
system.c	Stub file. Must be replaced by all integrators.	This file is a stub. This file include some function. Function used to enter to supervisor mode.
startup.c	Stub file. Must be replaced by all integrators.	This file is a stub. This file include sone functions necessary initializations for RAM. Copy the vector table from ROM to RAM. Copy initialized data from ROM to RAM
exceptions.c	Stub file. Must be replaced by all integrators.	This file is a stub. This file necessary handler exception when running application.
startup_↔ cm7.s	Stub file. Must be replaced by all integrators.	This file is a stub. This file need to run before initiatial application, it start-up code shall initialize the base addresses for interrupt and trap vector tables.
Vector_↔ Table.s	Stub file. Must be replaced by all integrators.	This file is a stub. This file necessary initializations for vector table.

3.3 Hardware Resources

#	Hardware IP	s32g274a	s32g254a	s32g233a	s32g234m	s32r45
1	NVIC	3	3	1	3	3
2	MCM	3	3	1	3	3

3.4 Deviations from Requirements

None.

3.5 Driver Limitations

The PLATFORM driver software have some following limitations for RTD S32 :

- Only one precompile configuration variant supported in the configuration tool
- No support for GPR registers

3.6 Driver usage and configuration tips

Platform driver does not support startup code and linker script, but it contains a sample for startup and linker. It also contains MPU and Cache Initialization, user should synchronize and disable cache before enable MPU, Platform also supports `sys_m7_cache_disable` and `sys_m7_cache_clean` functions in `system.c` file so user can refer to it for your project, please note that MPU and Cache enablement in startup code is just demo code, user can find detail about MPU support in RM driver and Cache support in MCL driver. Here is some samples for configuration we can custom for startup:

- User can define processor `-DD_CACHE_ENABLE` and `-DI_CACHE_ENABLE` to enable Dcache and Icache at startup code.
- User can enable MPU default configuration from startup code by using preprocessor `-DMPU_ENABLE`.
- MPU need to be enabled prior to Cache enablement, make sure `ENABLE_MPU` and `D_CACHE_ENABLE` `I_CACHE_ENABLE` are defined together.

The PLATFORM driver should generally be initialized before calling other software that requires platform specific setup, like interrupts configuration.

For M7 cores, the linker file provided DTCM sections. So, the user can allocate memory to these sections. Example for GCC compiler: `char attribute ((.dtcm_data)) Buffer[] = "DTCM DATA";`

3.6.1 Initialization The driver configuration contains a list of all implemented interrupt requests, with the associated settings. Besides that, it exposes information about all the configurable system-level settings, as well as interrupt monitors (if available). After generating the configuration structure from either Tresos or S32 Configuration Tool, the `Platform_Init` function should be called in order to apply the settings at NVIC level. Similarly, at the IP layer, the `IntCtrl_Ip_Init` function configures the entire list of configured interrupts at once, based on a structure generated by the tools.

3.6.2 Handling the interrupts If specific IRQs need to be configured alone, the dedicated API can be called to enable/disable, or set the priority for a single interrupt request (*Platform_SetIrq*, *Platform_SetIrqPriority*, and their equivalent at the *IntCtrl_Ip* level). The user can also optionally overwrite the default interrupt handler, by calling *Platform_InstallHandler* (or *IntCtrl_Ip_InstallHandler* at IP level).

The parameter for all interrupt-related APIs that identifies the interrupt request being handled is an enumeration called *IRQn_Type*, defined for each SoC in the platform header file.

3.7 Runtime errors

The driver does not generate DEM errors.

The development errors generated through DET are:

Error Code	Condition triggering the error
PLATFORM_E_PARAM_POINTER	Invalid pointer (null pointer) for parameters passed as reference
PLATFORM_E_PARAM_OUT_OF_RANGE	Parameter out of range
PLATFORM_E_VECTOR_TABLE_READ_ONLY	vector table resides in target flash
PLATFORM_E_PARAM_CONFIG	call from wrong mapped partition

3.8 Symbolic Names Disclaimer

All containers having *symbolicNameValue* set to TRUE in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Platform](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [PlatformDevErrorDetect](#)
 - * Parameter [PlatformMcmConfigurable](#)
 - * Parameter [PlatformIpAPIsAvailable](#)
 - * Parameter [PlatformEnableUserModeSupport](#)
 - * Parameter [PlatformMulticoreSupport](#)
 - * Parameter [PlatformEnableVtorConfiguration](#)
 - * Reference [PlatformEcucPartitionRef](#)
 - Container [McmConfig](#)
 - * Parameter [SystemAhbSlavePrio](#)
 - * Reference [PlatformMcmEcucPartitionRef](#)
 - * Container [SystemIsrConfig](#)
 - Parameter [SystemIsrName](#)
 - Parameter [SystemIsrEnabled](#)
 - Container [IntCtrlConfig](#)
 - * Parameter [PlatformVtorAddressConfig](#)
 - * Reference [PlatformNvicEcucPartitionRef](#)
 - * Container [PlatformIsrConfig](#)
 - Parameter [IsrName](#)
 - Parameter [IsrEnabled](#)
 - Parameter [IsrPriority](#)
 - Parameter [IsrHandler](#)
 - Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1 Module Platform

Configuration of Platform module.

Included containers:

- [GeneralConfiguration](#)
- [McmConfig](#)
- [IntCtrlConfig](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	False
supportedConfigVariants	VARIANT-PRE-COMPILE, VARIANT-POST-BUILD

4.2 Container GeneralConfiguration

GeneralConfiguration

This container contains the global configuration parameters of the Non-Autosar I2c driver.

Note: Implementation Specific Parameter.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter PlatformDevErrorDetect

PlatformDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	true

4.4 Parameter PlatformMcmConfigurable

PlatformMcmConfigurable

Check this in order to be able to configure Miscellaneous Control settings.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.5 Parameter PlatformIpAPIsAvailable

PlatformIpAPIsAvailable

Enable or disable IP layer APIs which are not used by APIs at High Level Driver (HLD). Following APIs are affected:

IntCtrl_Ip_SetPending

IntCtrl_Ip_GetPending

IntCtrl_Ip_GetActive

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.6 Parameter PlatformEnableUserModeSupport

When this parameter is enabled, the Platform module will adapt to run from User Mode, with the following measures:

b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.7 Parameter PlatformMulticoreSupport

This parameter globally enables the possibility to support multicore. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.8 Parameter PlatformEnableVtorConfiguration

When this parameter is enabled, the Platform module will allow the user to manually configure the Vector Table Offset Register address:

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.9 Reference PlatformEcucPartitionRef

Maps the Platform driver to zero a multiple ECUC partitions to make the modules API available in this partition.

Note: Each PlatformEcucPartitionRef should map to a M7 core, one by one

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.10 Container McmConfig

Vendor specific:

Miscellaneous Control Configuration

Included subcontainers:

- [SystemIsrConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.11 Parameter SystemAhbSlavePrio

Vendor specific:

Configures the access priority on the AHBS port of the Cortex-M7.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	Round_robin
literals	['Round_robin', 'AHB_Slave_priority']

4.12 Reference PlatformMcmEcucPartitionRef

Maps a instance of Mcm to ECUC partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.13 Container SystemIsrConfig

Vendor specific:

Configuration for core-related interrupts.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	7
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.14 Parameter SystemIsrName

Vendor specific:

Interrupt Name.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	FPU_INPUT_DENORMAL_IRQ
literals	['FPU_INPUT_DENORMAL_IRQ', 'FPU_INEXACT_IRQ', 'FPU_UNDE← RFLOW_IRQ', 'FPU_OVERFLOW_IRQ', 'FPU_DIVIDE_BY_ZERO_IRQ', 'FPU_INVALID_OPERATION_IRQ', 'TCM_WRITE_ABORT_IRQ']

4.15 Parameter SystemIsrEnabled

Vendor specific: Switch to indicate if the interrupt is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.16 Container IntCtrlConfig

Configuration for the interrupts.

Included subcontainers:

- [PlatformIsrConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.17 Parameter PlatformVtorAddressConfig

Configure the address where the Interrupt Vector starts

Property	Value
type	ECUC-INTEGGER-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.18 Reference PlatformNvicEcucPartitionRef

Maps an instance of Nvic ECUC partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.19 Container PlatformIsrConfig

Vendor specific:

Configuration for interrupt requests.

Warning: This is a precompile configuration.If you uncheck a ISR, you will not be able to enable the respective channel or error functionality at post build time.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	195
upperMultiplicity	195
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.20 Parameter IsrName

Vendor specific:

Interrupt Name.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	MSCM_Pcie_1_MSI_IRQn

Property	Value
literals	['MSCM_Pcie_1_MSI_IRQn', 'MSCM_INT0_IRQn', 'MSCM_INT1_IRQn', 'MSCM_INT2_IRQn', 'MSCM_Pcie_0_MSI_IRQn', 'CTI_INT0_IRQn', 'C← TI_INT1_IRQn', 'MCM_IRQn', 'DMA0_0_15_IRQn', 'DMA0_16_31_I← RQn', 'DMA0_ERR0_IRQn', 'DMA1_0_15_IRQn', 'DMA1_16_31_IRQn', 'DMA1_ERR0_IRQn', 'SWT0_IRQn', 'SWT1_IRQn', 'SWT2_IRQn', 'S← WT3_IRQn', 'SWT4_IRQn', 'SWT5_IRQn', 'SWT6_IRQn', 'SWT7_IRQn', 'MSCM_INT3_IRQn', 'MSCM_INT4_IRQn', 'STM0_IRQn', 'STM1_IRQn', 'STM2_IRQn', 'STM3_IRQn', 'STM4_IRQn', 'STM5_IRQn', 'STM6_IRQn', 'STM7_IRQn', 'QSPI0_IRQn', 'QSPI1_IRQn', 'QSPI2_IRQn', 'STCU2_LB← IST_MBIST_IRQn', 'USDHC_IRQn', 'CAN0_ORED_IRQn', 'CAN0_ERR← _IRQn', 'CAN0_ORED_0_7_MB_IRQn', 'CAN0_ORED_8_127_MB_IR← Qn', 'CAN1_ORED_IRQn', 'CAN1_ERR_IRQn', 'CAN1_ORED_0_7_MB← _IRQn', 'CAN1_ORED_8_127_MB_IRQn', 'CAN2_ORED_IRQn', 'CAN2← ERR_IRQn', 'CAN2_ORED_0_7_MB_IRQn', 'CAN2_ORED_8_127_M← B_IRQn', 'CAN3_ORED_IRQn', 'CAN3_ERR_IRQn', 'CAN3_ORED_0_← 7_MB_IRQn', 'CAN3_ORED_8_127_MB_IRQn', 'PIT0_IRQn', 'PIT1_IR← Qn', 'FTM0_IRQn', 'FTM1_IRQn', 'GMAC0_Common_IRQn', 'GMAC0_C← H0_TX_IRQn', 'GMAC0_CH0_RX_IRQn', 'GMAC0_CH1_TX_IRQn', 'G← MAC0_CH1_RX_IRQn', 'GMAC0_CH2_TX_IRQn', 'GMAC0_CH2_RX_← _IRQn', 'GMAC0_CH3_TX_IRQn', 'GMAC0_CH3_RX_IRQn', 'GMAC0_C← H4_TX_IRQn', 'GMAC0_CH4_RX_IRQn', 'MSCM_INT5_IRQn', 'MSCM← _INT6_IRQn', 'SAR_ADC0_INT_IRQn', 'SAR_ADC1_INT_IRQn', 'FLE← XRAY0_NCERR_IRQn', 'FLEXRAY0_CERR_IRQn', 'FLEXRAY0_CH0_← RX_FIFO_IRQn', 'FLEXRAY0_CH1_RX_FIFO_IRQn', 'FLEXRAY0_W← KUP_IRQn', 'FLEXRAY0_STATUS_IRQn', 'FLEXRAY0_CMBERR_IRQn', 'FLEXRAY0_TX_BUFF_IRQn', 'FLEXRAY0_RX_BUFF_IRQn', 'FLEX← RAY0_MODULE_IRQn', 'LINFLEXD0_IRQn', 'LINFLEXD1_IRQn', 'LIN← FLEXD2_IRQn', 'SPI0_IRQn', 'SPI1_IRQn', 'SPI2_IRQn', 'SPI3_IRQn', 'S← PI4_IRQn', 'SPI5_IRQn', 'I2C0_IRQn', 'I2C1_IRQn', 'I2C2_IRQn', 'I2C3← _IRQn', 'I2C4_IRQn', 'MC_RGM_IRQn', 'FCCU_ALARM_IRQn', 'FCCU← _MISC_IRQn', 'SBSW_IRQn', 'HSE_MU0_TX_IRQn', 'HSE_MU0_RX_← _IRQn', 'HSE_MU0_ORED_IRQn', 'HSE_MU1_TX_IRQn', 'HSE_MU1_R← X_IRQn', 'HSE_MU1_ORED_IRQn', 'HSE_MU2_TX_IRQn', 'HSE_MU2← _RX_IRQn', 'HSE_MU2_ORED_IRQn', 'HSE_MU3_TX_IRQn', 'HSE_← MU3_RX_IRQn', 'HSE_MU3_ORED_IRQn', 'DDR0_SCRUB_IRQn', 'DD← R0_PHY_IRQn', 'CTU_FIFO_FULL_EMPTY_IRQn', 'CTU_M_RELOA← D_IRQn', 'CTU_ERR_IRQn', 'TMU_ALARM_IRQn', 'RTC_SYS_CONT← _IRQn', 'PCIE0_ORED_DMA_IRQn', 'PCIE0_LINK_IRQn', 'PCIE0_AXI← _MSI_IRQn', 'PCIE0_PHY_DOWM_IRQn', 'PCIE0_PHY_UP_IRQn', 'P← CIE0_INTA_IRQn', 'PCIE0_INTB_IRQn', 'PCIE0_INTC_IRQn', 'PCIE0← _INTD_IRQn', 'PCIE0_MISC_IRQn', 'PCIE0_PCS_IRQn', 'PCIE0_TLP_← NC_IRQn', 'CORTEX_A53_ERR_L2RAM_CLUSTER0_IRQn', 'CORTEX← _A53_ERR_LIVLOCK_CLUSTER0_IRQn', 'CORTEX_A53_ERR_L2RA← M_CLUSTER1_IRQn', 'CORTEX_A53_ERR_LIVLOCK_CLUSTER1_IR← Qn', 'JDC_IRQn', 'SWT8_IRQn', 'SWT9_IRQn', 'SWT10_IRQn', 'SWT11← _IRQn', 'STM8_IRQn', 'STM9_IRQn', 'STM10_IRQn', 'STM11_IRQn', 'M← CSCM_INT7_IRQn', 'MCSCM_INT8_IRQn', 'MCSCM_INT9_IRQn', 'MC← SCM_INT10_IRQn', 'MCSCM_INT11_IRQn', 'LLCE0_INT0_IRQn', 'LLC← E0_INT1_IRQn', 'LLCE0_INT2_IRQn', 'LLCE0_INT3_IRQn', 'LLCE0_I← CSR14_IRQn', 'LLCE0_ICSR15_IRQn', 'LLCE0_ICSR16_IRQn', 'LLCE0← _ICSR17_IRQn', 'LLCE0_ICSR18_IRQn', 'LLCE0_ICSR19_IRQn', 'LLC← E0_ICSR20_IRQn', 'LLCE0_ICSR21_IRQn', 'LLCE0_ICSR22_IRQn', 'LL← CE0_ICSR23_IRQn', 'LLCE0_ICSR24_IRQn', 'LLCE0_ICSR25_IRQn', 'L← LCE0_ICSR26_IRQn', 'LLCE0_ICSR27_IRQn', 'PFE0_CH0_STAT_IRQn', 'PFE0_CH1_STAT_IRQn', 'PFE0_CH2_STAT_IRQn', 'PFE0_CH3_STA← T_IRQn', 'PFE0_BMUI_BMU2_IRQn', 'PFE0_HIF_NC_IRQn', 'PFE0← UT_GPT_IRQn', 'PFE0_PMT_IRQn', 'PFE0_ORED_IRQn', 'STM_TS_← CH_BEQ_IRQn', 'SHU1_ORED_IRQn', 'USB0_OTG_CORE_IRQn', 'IU←
22	S34 PLATFORM Driver NXP Semiconductors

Property	Value
----------	-------

4.21 Parameter IsrEnabled

Vendor specific: Switch to indicate if the interrupt is enabled

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.22 Parameter IsrPriority

Priority of the interrupt interrupt

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	0
max	15
min	0

4.23 Parameter IsrHandler

Function to be installed as the interrupt handler.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	undefined_handler

4.24 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.25 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	4
max	4
min	4

4.26 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	4
max	4
min	4

4.27 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.28 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	255
max	255
min	255

4.29 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	4
max	4
min	4

4.30 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	0
max	0
min	0

4.31 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	2
max	2
min	2

4.32 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

`<ModuleName>_<VendorId>_<VendorApiInfix>`.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name `Can_Write` defined in the SWS will translate to `Can_123_v11r456Write`.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	

4.33 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	43
max	43
min	43

This chapter describes the Tresos configuration plug-in for the *driver* Driver. The most of the parameters are described below.



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Platform	41
Interrupt Controller IP	31
System IP	50

Chapter 6

Module Documentation

6.1 Interrupt Controller IP

6.1.1 Detailed Description

Data Structures

- struct [IntCtrl_Ip_IrqConfigType](#)
Structure storing the state and priority configuration for an interrupt request. [More...](#)
- struct [IntCtrl_Ip_CtrlConfigType](#)
Structure storing the list of state configurations for all configured interrupts. [More...](#)

Types Reference

- typedef void(* [IntCtrl_Ip_IrqHandlerType](#)) (void)
Interrupt handler type.

Enum Reference

- enum [IntCtrl_Ip_StatusType](#)
Enumeration listing the possible error codes returned by IntCtrl_Ip API.
- enum [IntCtrl_Ip_IrqTargetType](#)
Enumeration listing the available target cores for an inter-core interrupt.

Function Reference

- [IntCtrl_Ip_StatusType](#) [IntCtrl_Ip_Init](#) (const [IntCtrl_Ip_CtrlConfigType](#) *pIntCtrlCtrlConfig)
Initializes the configured interrupts at interrupt controller level.
- void [IntCtrl_Ip_InstallHandler](#) (IRQn_Type eIrqNumber, const [IntCtrl_Ip_IrqHandlerType](#) pfNewHandler, [IntCtrl_Ip_IrqHandlerType](#) *const pfOldHandler)
Installs a handler for an IRQ.
- void [IntCtrl_Ip_EnableIrq](#) (IRQn_Type eIrqNumber)
Enables an interrupt request.
- void [IntCtrl_Ip_DisableIrq](#) (IRQn_Type eIrqNumber)
Disables an interrupt request.
- void [IntCtrl_Ip_SetPriority](#) (IRQn_Type eIrqNumber, uint8 u8Priority)
Sets the priority for an interrupt request.
- uint8 [IntCtrl_Ip_GetPriority](#) (IRQn_Type eIrqNumber)
Gets the priority for an interrupt request.
- void [IntCtrl_Ip_ClearPending](#) (IRQn_Type eIrqNumber)
Clears the pending flag for an interrupt request.
- void [IntCtrl_Ip_ClearDirectedCpuInterrupt](#) (IRQn_Type eIrqNumber)
Clear directed cpu Interrupt interrupt flag.
- boolean [IntCtrl_Ip_GetDirectedCpuInterrupt](#) (IRQn_Type eIrqNumber)
Get directed cpu Interrupt interrupt flag.
- void [IntCtrl_Ip_GenerateDirectedCpuInterrupt](#) (IRQn_Type eIrqNumber, [IntCtrl_Ip_IrqTargetType](#) eCpuTarget)
Generates an interrupt request to a CPU target.

6.1.2 Data Structure Documentation

6.1.2.1 struct IntCtrl_Ip_IrqConfigType

Structure storing the state and priority configuration for an interrupt request.

Definition at line 111 of file IntCtrl_Ip_TypesDef.h.

Data Fields

- IRQn_Type [eIrqNumber](#)
Interrupt number.
- boolean [bIrqEnabled](#)
Interrupt state (enabled/disabled)
- uint8 [u8IrqPriority](#)
Interrupt priority.
- [IntCtrl_Ip_IrqHandlerType](#) [pfHandler](#)
Interrupt handler.

6.1.2.1.1 Field Documentation

6.1.2.1.1.1 **eIrqNumber** `IRQn_Type eIrqNumber`

Interrupt number.

Definition at line 114 of file `IntCtrl_Ip_TypesDef.h`.

6.1.2.1.1.2 **bIrqEnabled** `boolean bIrqEnabled`

Interrupt state (enabled/disabled)

Definition at line 116 of file `IntCtrl_Ip_TypesDef.h`.

6.1.2.1.1.3 **u8IrqPriority** `uint8 u8IrqPriority`

Interrupt priority.

Definition at line 118 of file `IntCtrl_Ip_TypesDef.h`.

6.1.2.1.1.4 **pfHandler** `IntCtrl_Ip_IrqHandlerType pfHandler`

Interrupt handler.

Definition at line 120 of file `IntCtrl_Ip_TypesDef.h`.

6.1.2.2 **struct IntCtrl_Ip_CtrlConfigType**

Structure storing the list of state configurations for all configured interrupts.

Definition at line 127 of file `IntCtrl_Ip_TypesDef.h`.

Data Fields

- `uint32 u32ConfigIrqCount`
Number of configured interrupts.
- `const IntCtrl_Ip_IrqConfigType * aIrqConfig`
List of interrupts configurations.

6.1.2.2.1 Field Documentation

6.1.2.2.1.1 u32ConfigIrqCount `uint32 u32ConfigIrqCount`

Number of configured interrupts.

Definition at line 130 of file `IntCtrl_Ip_TypesDef.h`.

6.1.2.2.1.2 aIrqConfig `const IntCtrl_Ip_IrqConfigType* aIrqConfig`

List of interrupts configurations.

Definition at line 136 of file `IntCtrl_Ip_TypesDef.h`.

6.1.3 Types Reference

6.1.3.1 IntCtrl_Ip_IrqHandlerType

```
typedef void(* IntCtrl_Ip_IrqHandlerType) (void)
```

Interrupt handler type.

Definition at line 77 of file `IntCtrl_Ip_TypesDef.h`.

6.1.4 Enum Reference

6.1.4.1 IntCtrl_Ip_StatusType

```
enum IntCtrl_Ip_StatusType
```

Enumeration listing the possible error codes returned by `IntCtrl_Ip` API.

Enumerator

<code>INTCTRL_IP_STATUS_SUCCESS</code>	Status SUCCESS.
<code>INTCTRL_IP_STATUS_ERROR</code>	Status ERROR.

Definition at line 143 of file `IntCtrl_Ip_TypesDef.h`.

6.1.4.2 IntCtrl_Ip_IrqTargetType

```
enum IntCtrl_Ip_IrqTargetType
```

Enumeration listing the available target cores for an inter-core interrupt.

Enumerator

INTCTRL_IP_TARGET_SELF	Interrupt request targeted to the same core that triggers it.
INTCTRL_IP_TARGET_OTHERS	Interrupt request targeted to all the other cores.
INTCTRL_IP_TARGET_CP0	Interrupt request targeted to core 0.
INTCTRL_IP_TARGET_CP1	Interrupt request targeted to core 1.
INTCTRL_IP_TARGET_CP2	Interrupt request targeted to core 2.
INTCTRL_IP_TARGET_CP3	Interrupt request targeted to core 3.
INTCTRL_IP_TARGET_CP4	Interrupt request targeted to core 4.
INTCTRL_IP_TARGET_CP5	Interrupt request targeted to core 5.
INTCTRL_IP_TARGET_CP6	Interrupt request targeted to core 6.

Definition at line 157 of file IntCtrl_Ip_TypesDef.h.

6.1.5 Function Reference

6.1.5.1 IntCtrl_Ip_Init()

```
IntCtrl_Ip_StatusType IntCtrl_Ip_Init (
    const IntCtrl_Ip_CtrlConfigType * pIntCtrlCtrlConfig )
```

Initializes the configured interrupts at interrupt controller level.

This function is non-reentrant and initializes the interrupts.

Parameters

in	<i>pIntCtrlCtrlConfig</i>	pointer to configuration structure for interrupts.
----	---------------------------	--

Returns

IntCtrl_Ip_StatusType: error code.

6.1.5.2 IntCtrl_Ip_InstallHandler()

```
void IntCtrl_Ip_InstallHandler (
    IRQn_Type eIrqNumber,
    const IntCtrl_Ip_IrqHandlerType pfNewHandler,
    IntCtrl_Ip_IrqHandlerType *const pfOldHandler )
```

Installs a handler for an IRQ.

This function is non-reentrant; it installs an new ISR for an interrupt line.

Note

This function works only when the interrupt vector table resides in RAM.

Parameters

in	<i>eIrqNumber</i>	interrupt number.
in	<i>pfNewHandler</i>	function pointer for the new handler.
out	<i>pfOldHandler</i>	stores the address of the old interrupt handler.

Returns

void.

6.1.5.3 IntCtrl_Ip_EnableIrq()

```
void IntCtrl_Ip_EnableIrq (
    IRQn_Type eIrqNumber )
```

Enables an interrupt request.

This function is non-reentrant; it enables the interrupt request at interrupt controller level.

Parameters

in	<i>eIrqNumber</i>	interrupt number to be enabled.
----	-------------------	---------------------------------

Returns

void.

6.1.5.4 IntCtrl_Ip_DisableIrq()

```
void IntCtrl_Ip_DisableIrq (
    IRQn_Type eIrqNumber )
```

Disables an interrupt request.

This function is non-reentrant; it disables the interrupt request at interrupt controller level.

Parameters

in	<i>eIrqNumber</i>	interrupt number to be disabled.
----	-------------------	----------------------------------

Returns

void.

6.1.5.5 IntCtrl_Ip_SetPriority()

```
void IntCtrl_Ip_SetPriority (
    IRQn_Type eIrqNumber,
    uint8 u8Priority )
```

Sets the priority for an interrupt request.

This function is non-reentrant; it sets the priority for the interrupt request.

Parameters

in	<i>eIrqNumber</i>	interrupt number for which the priority is set.
in	<i>u8Priority</i>	the priority to be set.

Returns

void.

6.1.5.6 IntCtrl_Ip_GetPriority()

```
uint8 IntCtrl_Ip_GetPriority (
    IRQn_Type eIrqNumber )
```

Gets the priority for an interrupt request.

This function is non-reentrant; it retrieves the priority for the interrupt request.

Parameters

in	<i>eIrqNumber</i>	interrupt number for which the priority is set.
----	-------------------	---

Returns

uint8: the priority of the interrupt.

6.1.5.7 IntCtrl_Ip_ClearPending()

```
void IntCtrl_Ip_ClearPending (
    IRQn_Type eIrqNumber )
```

Clears the pending flag for an interrupt request.

This function is reentrant; it clears the pending flag for the interrupt request.

Parameters

in	<i>eIrqNumber</i>	interrupt number for which the pending flag is cleared.
----	-------------------	---

Returns

void.

6.1.5.8 IntCtrl_Ip_ClearDirectedCpuInterrupt()

```
void IntCtrl_Ip_ClearDirectedCpuInterrupt (
    IRQn_Type eIrqNumber )
```

Clear directed cpu Interrupt interrupt flag.

This function is non-reentrant; it is provided for clearing directed cpu Interrupt interrupt flag.

Parameters

in	<i>eIrqNumber</i>	interrupt number
----	-------------------	------------------

Returns

void.

6.1.5.9 IntCtrl_Ip_GetDirectedCpuInterrupt()

```
boolean IntCtrl_Ip_GetDirectedCpuInterrupt (
    IRQn_Type eIrqNumber )
```

Get directed cpu Interrupt interrupt flag.

This function is non-reentrant; it is provided for getting directed cpu Interrupt interrupt flag.

Parameters

in	<i>eIrqNumber</i>	interrupt number
----	-------------------	------------------

Returns

boolean: TRUE - flag set, FALSE - flag cleared.

6.1.5.10 IntCtrl_Ip_GenerateDirectedCpuInterrupt()

```
void IntCtrl_Ip_GenerateDirectedCpuInterrupt (
    IRQn_Type eIrqNumber,
    IntCtrl_Ip_IrqTargetType eCpuTarget )
```

Generates an interrupt request to a CPU target.

This function is non-reentrant; it is provided for generating a directed interrupt to a CPU defined by target parameter.

Parameters

in	<i>eIrqNumber</i>	interrupt number to be triggered.
in	<i>eCpuTarget</i>	target core for the interrupt request.

Returns

void.

6.2 Platform

6.2.1 Detailed Description

Modules

- [Interrupt Controller IP](#)
- [System IP](#)

Data Structures

- struct [Platform_ConfigType](#)
Configuration structure for PLATFORM CDD. [More...](#)

Macros

- `#define PLATFORM_E_PARAM_POINTER`
All API's having pointers as parameters shall return this error if called with a NULL value.
- `#define PLATFORM_E_PARAM_OUT_OF_RANGE`
Error returned for parameters out of range.
- `#define PLATFORM_E_PARAM_CONFIG`
If DET error reporting is enabled, the PLATFORM will check upon each API call if the requested resource is configured to be available on the current core, and in case of error will return PLATFORM_E_PARAM_CONFIG.
- `#define PLATFORM_INIT_ID`
Service ID of Platform_Init function.
- `#define PLATFORM_SET_IRQ_ID`
Service ID of Platform_SetIrq function.
- `#define PLATFORM_SET_IRQ_PRIO_ID`
Service ID of Platform_SetIrqPriority function.
- `#define PLATFORM_GET_IRQ_PRIO_ID`
Service ID of Platform_GetIrqPriority function.
- `#define PLATFORM_INSTALL_HANDLER_ID`
Service ID of Platform_InstallIrqHandler function.
- `#define PLATFORM_SET_IRQ_MONITOR_ID`
Service ID of Platform_SetIrqMonitor function.
- `#define PLATFORM_ACK_IRQ_ID`
Service ID of Platform_AckIrq function.
- `#define PLATFORM_SELECT_MONITORED_IRQ_ID`
Service ID of Platform_SelectMonitoredIrq function.
- `#define PLATFORM_SET_MONITORED_IRQ_LATENCY_ID`
Service ID of Platform_SetMonitoredIrqLatency function.
- `#define PLATFORM_RESET_IRQ_MONITOR_TIMER_ID`
Service ID of Platform_ResetIrqMonitorTimer function.
- `#define PLATFORM_GET_IRQ_MONITOR_STATUS_ID`
Service ID of Platform_GetIrqMonitorStatus function.

Types Reference

- typedef [IntCtrl_Ip_IrqHandlerType](#) [Platform_IrqHandlerType](#)
Interrupt handler type definition for PLATFORM CDD.

Function Reference

- void [Platform_Init](#) (const [Platform_ConfigType](#) *pConfig)
Initializes the platform settings based on user configuration.
- Std_ReturnType [Platform_SetIrq](#) (IRQn_Type eIrqNumber, boolean bEnable)
Configures (enables/disables) an interrupt request.
- Std_ReturnType [Platform_SetIrqPriority](#) (IRQn_Type eIrqNumber, uint8 u8Priority)
Configures the priority of an interrupt request.
- Std_ReturnType [Platform_GetIrqPriority](#) (IRQn_Type eIrqNumber, uint8 *u8Priority)
Returns the priority of an interrupt request.
- Std_ReturnType [Platform_InstallIrqHandler](#) (IRQn_Type eIrqNumber, const [Platform_IrqHandlerType](#) pfNewHandler, [Platform_IrqHandlerType](#) *const pfOldHandler)
Installs a new handler for an interrupt request.

6.2.2 Data Structure Documentation

6.2.2.1 struct Platform_ConfigType

Configuration structure for PLATFORM CDD.

Definition at line 174 of file Platform_TypesDef.h.

Data Fields

- const Platform_Ipw_ConfigType * [pIpwConfig](#)
Reference to IPW structure.

6.2.2.1.1 Field Documentation

6.2.2.1.1.1 [pIpwConfig](#) const Platform_Ipw_ConfigType* pIpwConfig

Reference to IPW structure.

Definition at line 177 of file Platform_TypesDef.h.

6.2.3 Macro Definition Documentation

6.2.3.1 PLATFORM_E_PARAM_POINTER

```
#define PLATFORM_E_PARAM_POINTER
```

All API's having pointers as parameters shall return this error if called with with a NULL value.

Definition at line 84 of file Platform_TypesDef.h.

6.2.3.2 PLATFORM_E_PARAM_OUT_OF_RANGE

```
#define PLATFORM_E_PARAM_OUT_OF_RANGE
```

Error returned for parameters out of range.

Definition at line 90 of file Platform_TypesDef.h.

6.2.3.3 PLATFORM_E_PARAM_CONFIG

```
#define PLATFORM_E_PARAM_CONFIG
```

If DET error reporting is enabled, the PLATFORM will check upon each API call if the requested resource is configured to be available on the current core, and in case of error will return PLATFORM_E_PARAM_CONFIG.

Definition at line 99 of file Platform_TypesDef.h.

6.2.3.4 PLATFORM_INIT_ID

```
#define PLATFORM_INIT_ID
```

Service ID of Platform_Init function.

Parameter used when raising an error/exception

Definition at line 105 of file Platform_TypesDef.h.

6.2.3.5 PLATFORM_SET_IRQ_ID

```
#define PLATFORM_SET_IRQ_ID
```

Service ID of Platform_SetIrq function.

Parameter used when raising an error/exception

Definition at line 111 of file Platform_TypesDef.h.

6.2.3.6 PLATFORM_SET_IRQ_PRIO_ID

```
#define PLATFORM_SET_IRQ_PRIO_ID
```

Service ID of Platform_SetIrqPriority function.

Parameter used when raising an error/exception

Definition at line 117 of file Platform_TypesDef.h.

6.2.3.7 PLATFORM_GET_IRQ_PRIO_ID

```
#define PLATFORM_GET_IRQ_PRIO_ID
```

Service ID of Platform_GetIrqPriority function.

Parameter used when raising an error/exception

Definition at line 123 of file Platform_TypesDef.h.

6.2.3.8 PLATFORM_INSTALL_HANDLER_ID

```
#define PLATFORM_INSTALL_HANDLER_ID
```

Service ID of Platform_InstallIrqHandler function.

Parameter used when raising an error/exception

Definition at line 129 of file Platform_TypesDef.h.

6.2.3.9 PLATFORM_SET_IRQ_MONITOR_ID

```
#define PLATFORM_SET_IRQ_MONITOR_ID
```

Service ID of Platform_SetIrqMonitor function.

Parameter used when raising an error/exception

Definition at line 135 of file Platform_TypesDef.h.

6.2.3.10 PLATFORM_ACK_IRQ_ID

```
#define PLATFORM_ACK_IRQ_ID
```

Service ID of Platform_AckIrq function.

Parameter used when raising an error/exception

Definition at line 141 of file Platform_TypesDef.h.

6.2.3.11 PLATFORM_SELECT_MONITORED_IRQ_ID

```
#define PLATFORM_SELECT_MONITORED_IRQ_ID
```

Service ID of Platform_SelectMonitoredIrq function.

Parameter used when raising an error/exception

Definition at line 147 of file Platform_TypesDef.h.

6.2.3.12 PLATFORM_SET_MONITORED_IRQ_LATENCY_ID

```
#define PLATFORM_SET_MONITORED_IRQ_LATENCY_ID
```

Service ID of Platform_SetMonitoredIrqLatency function.

Parameter used when raising an error/exception

Definition at line 153 of file Platform_TypesDef.h.

6.2.3.13 PLATFORM_RESET_IRQ_MONITOR_TIMER_ID

```
#define PLATFORM_RESET_IRQ_MONITOR_TIMER_ID
```

Service ID of Platform_ResetIrqMonitorTimer function.

Parameter used when raising an error/exception

Definition at line 159 of file Platform_TypesDef.h.

6.2.3.14 PLATFORM_GET_IRQ_MONITOR_STATUS_ID

```
#define PLATFORM_GET_IRQ_MONITOR_STATUS_ID
```

Service ID of Platform_GetIrqMonitorStatus function.

Parameter used when raising an error/exception

Definition at line 165 of file Platform_TypesDef.h.

6.2.4 Types Reference

6.2.4.1 Platform_IrqHandlerType

```
typedef IntCtrl_Ip_IrqHandlerType Platform_IrqHandlerType
```

Interrupt handler type definition for PLATFORM CDD.

Definition at line 188 of file Platform_TypesDef.h.

6.2.5 Function Reference

6.2.5.1 Platform_Init()

```
void Platform_Init (
    const Platform_ConfigType * pConfig )
```

Initializes the platform settings based on user configuration.

This function is non-reentrant; it initializes the interrupts, interrupt monitors (if available), as well as other platform specific settings as defined for each SoC.

Parameters

in	<i>pConfig</i>	pointer to platform configuration structure.
----	----------------	--

Returns

void

6.2.5.2 Platform_SetIrq()

```
Std_ReturnType Platform_SetIrq (
    IRQn_Type eIrqNumber,
    boolean bEnable )
```

Configures (enables/disables) an interrupt request.

This function is non-reentrant; it enables/disables the selected interrupt.

Parameters

in	<i>eIrqNumber</i>	interrupt to be configured.
in	<i>bEnable</i>	TRUE - enable interrupt, FALSE - disable interrupt.

Returns

Std_ReturnType: E_OK/E_NOT_OK; specific errors are reported through DET.

6.2.5.3 Platform_SetIrqPriority()

```
Std_ReturnType Platform_SetIrqPriority (
    IRQn_Type eIrqNumber,
    uint8 u8Priority )
```

Configures the priority of an interrupt request.

This function is non-reentrant; it sets the priority for the selected interrupt.

Parameters

in	<i>eIrqNumber</i>	interrupt number for which priority is configured.
in	<i>u8Priority</i>	desired priority of the interrupt.

Returns

Std_ReturnType: E_OK/E_NOT_OK; specific errors are reported through DET.

6.2.5.4 Platform_GetIrqPriority()

```
Std_ReturnType Platform_GetIrqPriority (
    IRQn_Type eIrqNumber,
    uint8 * u8Priority )
```

Returns the priority of an interrupt request.

This function is non-reentrant; it retrieves the current priority of the selected interrupt.

Parameters

in	<i>eIrqNumber</i>	interrupt number for which priority is returned.
out	<i>u8Priority</i>	output parameter storing the priority of the interrupt.

Returns

Std_ReturnType: E_OK/E_NOT_OK; specific errors are reported through DET.

6.2.5.5 Platform_InstallIrqHandler()

```
Std_ReturnType Platform_InstallIrqHandler (
    IRQn_Type eIrqNumber,
    const Platform_IrqHandlerType pfNewHandler,
    Platform_IrqHandlerType *const pfOldHandler )
```

Installs a new handler for an interrupt request.

This function is non-reentrant; it replaces the current interrupt handler for the selected interrupt with the new function provided as the second parameter. The address of the old handler can be optionally stored in the third parameter.

Parameters

in	<i>eIrqNumber</i>	interrupt number for which priority is returned.
in	<i>pfNewHandler</i>	function pointer for the new handler.
out	<i>pfOldHandler</i>	function pointer that will store the address of the old handler

Note

- this parameter can be passed as NULL if not needed.

Returns

pfOldHandler: E_OK/E_NOT_OK; specific errors are reported through DET.

6.3 System IP

6.3.1 Detailed Description

Function Reference

- void [System_Ip_SetAhbSlavePriority](#) (boolean bPriority)
Selects the access priority on the AHBS port of the Cortex-M7.
- void [System_Ip_ConfigIrq](#) (System_Ip_IrqType eIrq, boolean bEnable)
Enables/disables core-related interrupt exceptions.
- void [System_Ip_ClearWriteAbortFlag](#) (void)
Clears Write Abort on Slave flag.
- uint32 [System_Ip_GetPlatformRevision](#) (void)
Returns platform revision.

6.3.2 Function Reference

6.3.2.1 System_Ip_SetAhbSlavePriority()

```
void System_Ip_SetAhbSlavePriority (
    boolean bPriority )
```

Selects the access priority on the AHBS port of the Cortex-M7.

This function is non-reentrant and configures the AHB slave priority.

Parameters

in	<i>bPriority</i>	FALSE - round-robin arbitration scheme, TRUE - AHB-slave access has priority over a core access
----	------------------	---

Returns

void

6.3.2.2 System_Ip_ConfigIrq()

```
void System_Ip_ConfigIrq (
    System_Ip_IrqType eIrq,
    boolean bEnable )
```

Enables/disables core-related interrupt exceptions.

This function is non-reentrant and configures core-related interrupt exceptions, as defined per each platform.

Parameters

in	<i>eIrq</i>	core-related interrupt event.
in	<i>bEnable</i>	FALSE - disable interrupt, TRUE - enable interrupt.

Returns

void

6.3.2.3 System_Ip_ClearWriteAbortFlag()

```
void System_Ip_ClearWriteAbortFlag (
    void )
```

Clears Write Abort on Slave flag.

This function is reentrant and clears the flag indicating when a write abort has occurred on the AHBS interface.

Returns

void

6.3.2.4 System_Ip_GetPlatformRevision()

```
uint32 System_Ip_GetPlatformRevision (
    void )
```

Returns platform revision.

This function is reentrant and returns a software-visible revision number.

Returns

void

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

